python-can-candle

English

`python-can-candle` 适合在 Windows 或 Linux 上使用 Python 直接访问 CAN 和 CAN FD 设备。

开发板包含两颗 MCU，每颗 MCU 提供两个 CAN/CAN FD channel，因此通常可 检测到两个 USB 设备、共四个 channel。USB 设备枚举顺序和 channel 编号不能 直接对应板上接口丝印，首次使用时应逐路确认。

`python-can-candle` 主要用于 Windows Python 程序控制，以及需要跨平台统一 `python-can` API 的场景。

安装

本说明基于 `python-can-candle 1.2.4`：

```
python -m pip install "python-can-candle==1.2.4"
```

SANPO 集成板已经通过 Windows WCID 自动绑定 WinUSB， 不需要使用 Zadig，也不需要安装驱动。

V81 及以前的固件版本 USB 产品名为 `SANPO gs_usb CDC Composite fdV8`，V82 及后续 固件使用简化后的 `SANPO SPINE`。本页提供的完整样例会同时识别这两个名称， 并在打开设备时使用实际枚举到的产品名，因此升级固件后不需要手动修改样例。 如果客户程序显式传入 `product=`，该值必须与当前固件报告的产品名一致；为了 为兼容 V81 以及 V82/V83等后续版本，下面的基础代码片段不限制 `product`。

安装后查看版本：

```
python -m pip show python-can-candle candle-api python-can
```

检测通道

```
import can

for config in can.detect_available_configs("candle"):
    print(config)
```

开发板通常输出四项，channel 格式类似：

```
SERIAL_0:0
SERIAL_0:1
SERIAL_1:0
SERIAL_1:1
```

`SERIAL_0`、`SERIAL_1` 是两颗 MCU 各自的 USB 序列号。序列号可以稳定区分 两颗 MCU，但不能仅凭序列号判断它对应板上的哪两路接口。

经典 CAN

以下示例在指定 MCU 的 channel 0 上使用 1 Mbit/s，发送一个标准帧并等待回复：

```
import can

serial = "替换为实际序列号"

with can.Bus(
    interface="candle",
    channel=f"{serial}:0",
    vid=0x1209,
    pid=0x2323,
    manufacture="SANPO",
    bitrate=1_000_000,
    sample_point=80.0,
    ignore_config=True,
) as bus:
    message = can.Message(
        arbitration_id=0x123,
        is_extended_id=False,
        data=[0x11, 0x22, 0x33, 0x44],
    )
    bus.send(message, timeout=1.0)
    reply = bus.recv(timeout=1.0)
    print(reply)
```

示例 ID 和数据没有实际设备含义，应替换为所连接设备的协议内容。

CAN FD

以下示例配置仲裁段 1 Mbit/s、数据段 5 Mbit/s，并发送开启 BRS 的 CAN FD 帧：

```
import can

serial = "替换为实际序列号"

with can.Bus(
    interface="candle",
    channel=f"{serial}:0",
    vid=0x1209,
    pid=0x2323,
    manufacture="SANPO",
    fd=True,
    bitrate=1_000_000,
    sample_point=80.0,
    data_bitrate=5_000_000,
    data_sample_point=75.0,
    ignore_config=True,
) as bus:
    message = can.Message(
        arbitration_id=0x123,
        is_extended_id=False,
        is_fd=True,
        bitrate_switch=True,
        data=bytes(range(16)),
    )
    bus.send(message, timeout=1.0)
```

SANPO SPINE 的 CAN FD 数据段支持 2/3/4/5 Mbit/s。使用 2/3/4 Mbit/s 时 推荐 `data_sample_point=80.0`；使用 5 Mbit/s 时推荐 `data_sample_point=75.0`。

同一颗 MCU 的两个通道

同一颗 MCU 的两个 channel 应由一个 `CandleBus` 实例共同打开：

```
import can

with can.Bus(
    interface="candle",
    channel=[0, 1],
    serial_number="替换为实际序列号",
    bitrate=1_000_000,
    sample_point=80.0,
    ignore_config=True,
) as bus:
    message = can.Message(
        arbitration_id=0x123,
        is_extended_id=False,
        channel=1,
        data=[0x01, 0x02],
    )
    bus.send(message, timeout=1.0)
```

这里的 `Message.channel` 只用于主机选择物理 CAN 通道，不会写入 CAN 数据， 也不是 SANPO 自定义通道报文。访问完整四路 CAN 时，应分别为两颗 MCU 创建 一个 `CandleBus` 。

小米 CyberGear 示例

示例程序： python-can-candle CyberGear 逐步使用说明

先执行只读设备枚举，不发送 CAN 帧：

```
python pythoncan_cybergear_demo_v8.py --list-devices
```

确认两颗 MCU 的序列号和物理通道映射后，再进行空载初始化测试。以下命令会 配置 CAN 并向电机发送初始化、使能和停止指令：

```
python pythoncan_cybergear_demo_v8.py \
  --enable-motion \
  --init-only \
  --device-serials "SERIAL_0,SERIAL_1" \
  --motor-map "0/0:1,2,3;0/1:4,5,6;1/0:7,8,9;1/1:10,11,12"
```

`0/1` 表示 `--device-serials` 中第 0 颗 MCU 的 channel 1。首次运行必须空载， 确认急停有效，并根据实际接线调整电机 ID 和通道映射。

使用限制

- 同一个 gs_usb 接口不能同时由 CANgaroo、`python-can-candle` 或其他 WinUSB 程序占用。运行 Python 示例前应关闭 CANgaroo 的连接。
- 同一个物理 CAN/CAN FD 通道不要同时由 USB 透传、管理口协议、SPI 或离线 任务发送数据。
- `python-can-candle` 打开通道时会重新配置波特率；这些设置不会作为用户配置 永久保存。
- Linux 下如果接口已经由内核 `gs_usb` 驱动生成 `canX`，应直接使用 SocketCAN，避免两个主机驱动竞争同一 USB 接口。

常见问题

现象	处理方法
`Access denied` 或设备忙	关闭 CANgaroo、其他 Python 进程及占用该 WinUSB 接口的软件
能发送但设备不回复	检查 channel 与物理接口映射、波特率、终端电阻、设备 ID 和供电
CAN FD 通信错误	确认仲裁段、数据段波特率及采样点与总线中所有设备一致